COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	10/06/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 11 **Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE:

mainA.cpp > Graph

```cpp
#include <iostream>
#include <vector>

struct Edge {
    int start;
    int end;
    int weight;
};

// Adjacency Matrix
void displayMatrix(const std::vector<Edge>& edges, int vertices) {
    int matrix[10][10] = {};

    for (const Edge& edge : edges) {
        matrix[edge.start][edge.end] = edge.weight;
    }

    std::cout << "\nADJACENCY MATRIX\n\n";

    std::cout << "    ";
    for (int i = 0; i < vertices; i++) {
        std::cout << i << "   ";
    }
    std::cout << "\n";

    for (int i = 0; i < vertices; i++) {
        std::cout << i << " | ";

        for (int j = 0; j < vertices; j++) {
            std::cout << matrix[i][j] << "   ";
        }

        std::cout << "\n";
    }
}

// Adjacency List
struct AdjNode {
    int value;
    int weight;
    AdjNode* next;
};

class Graph {
private:
    int vertices;

public:
    AdjNode** head;

    Graph(int vertices) {
        this->vertices = vertices;

        head = new AdjNode*[vertices];

        for (int i = 0; i < vertices; i++) {
            head[i] = nullptr;
        }
    }

    void addEdge(int start, int end, int weight) {
        AdjNode* newNode = new AdjNode;

        newNode->value = end;
        newNode->weight = weight;
        newNode->next = head[start];

        head[start] = newNode;
    }

```

```
71      void displayList() {
72          std::cout << "\nADJACENCY LIST\n\n";
73
74          for (int i = 0; i < vertices; i++) {
75              std::cout << i << " -> ";
76
77              AdjNode* current = head[i];
78
79              while (current != nullptr) {
80                  std::cout << "("
81                            << current->value
82                            << ", "
83                            << current->weight
84                            << ") ";
85
86                  current = current->next;
87              }
88
89              std::cout << "\n";
90          }
91      }
92
93      ~Graph() {
94          for (int i = 0; i < vertices; i++) {
95              AdjNode* current = head[i];
96
97              while (current != nullptr) {
98                  AdjNode* temp = current;
99                  current = current->next;
100                 delete temp;
101             }
102         }
103
104         delete[] head;
105     }
106 };
107
108 int main() {
109
110     // Graph edges from the activity
111     Edge edges[] = {
112         {0, 1, 2},
113         {0, 2, 4},
114         {1, 4, 3},
115         {2, 3, 2},
116         {3, 1, 4},
117         {4, 3, 3}
118     };
119
120     int vertices = 6;
121     int edgeCount = sizeof(edges) / sizeof(edges[0]);
122
123     // Adjacency Matrix
124     std::vector<Edge> edgeList;
125
126     for (int i = 0; i < edgeCount; i++) {
127         edgeList.push_back(edges[i]);
128     }
129
130     displayMatrix(edgeList, vertices);
131
132     // Adjacency List
133     Graph graph(vertices);
134
135     for (int i = 0; i < edgeCount; i++) {
136         graph.addEdge(
137             edges[i].start,
```

```
    for (int i = 0; i < edgeCount; i++) {
        graph.addEdge(
            edges[i].start,
            edges[i].end,
            edges[i].weight
        );
    }

    graph.displayList();

    return 0;
}
```

OUTPUT:

```
ADJACENCY MATRIX

    0   1   2   3   4   5
0 | 0   2   4   0   0   0
1 | 0   0   0   0   3   0
2 | 0   0   0   2   0   0
3 | 0   4   0   0   0   0
4 | 0   0   0   3   0   0
5 | 0   0   0   0   0   0

ADJACENCY LIST

0 -> (2, 4) (1, 2)
1 -> (4, 3)
2 -> (3, 2)
3 -> (1, 4)
4 -> (3, 3)
5 ->
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

CODE:

graph.h > ...

```cpp
#ifndef GRAPH_H
#define GRAPH_H

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
};

template <typename T>
class Graph
{
private:
    size_t V;
    std::vector<Edge<T>> edge_list;

public:
    Graph(size_t N) : V(N) {}

    void add_edge(Edge<T>&& e)
    {
        edge_list.push_back(e);
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;

        for (const auto& e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.push_back(e);
            }
        }

        return edges_from_v;
    }

    auto vertices() const
    {
        return V;
    }
};

template <typename T>
Graph<T> create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned,
    std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2,0}, {5,0}};
    edges[2] = {{1,0}, {5,0}, {4,0}};
    edges[3] = {{4,0}, {7,0}};
    edges[4] = {{2,0}, {3,0}, {5,0}, {6,0}, {8,0}};
    edges[5] = {{1,0}, {2,0}, {4,0}, {8,0}};
    edges[6] = {{4,0}, {7,0}, {8,0}};
    edges[7] = {{3,0}, {6,0}};
    edges[8] = {{4,0}, {5,0}, {6,0}};

    for (auto& i : edges)
    {
        for (auto& j : i.second)
        {
            G.add_edge(
                Edge<T>{i.first, j.first, j.second}
            );
```

```cpp
    }

    return G;
}

// B.1
template <typename T>
std::vector<size_t> depth_first_search(const Graph<T>& G)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

// B.2
template <typename T>
std::vector<size_t> breadth_first_search(const Graph<T>& G)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                queue.push(e.dest);
            }
        }
    }

    return visit_order;
}
```

B.1. Depth-First Search

CODE:

```cpp
// B.1
template <typename T>
std::vector<size_t> depth_first_search(const Graph<T>& G)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}
```

```cpp
int main(){

    // ILO B1
    Graph<unsigned> G =
        create_reference_graph<unsigned>();

    std::cout << "DFS Order of vertices:" << std::endl;
    auto dfs = depth_first_search(G);

    for (auto vertex : dfs)
    {
        std::cout << vertex << " ";
    }
```

OUTPUT:

```
DFS Order of vertices:
1 5 8 6 7 3 4 2
```

B.2. Breadth-First Search

CODE:

```cpp
// B.2
template <typename T>
std::vector<size_t> breadth_first_search(const Graph<T>& G)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                queue.push(e.dest);
            }
        }
    }

    return visit_order;
}
```

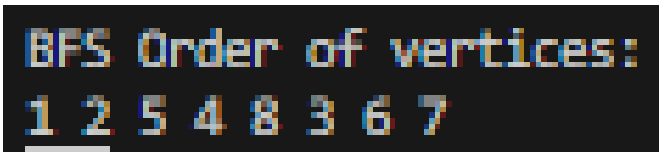
```cpp
    // ILO B2
    std::cout << "\n\n";
    std::cout << "BFS Order of vertices:" << std::endl;
    auto bfs = breadth_first_search(G);

    for (auto vertex : bfs)
    {
        std::cout << vertex << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

OUTPUT:



ANALYSIS:

Here in procedure activity, I learned how graphs can be implemented using an adjacency matrix and an adjacency list. The graph represents vertices and the connections or edges between them. I also learned how graph traversal works using Depth-First Search (DFS) and Breadth-First Search (BFS). DFS explores a path as deeply as possible before backtracking, while BFS visits the vertices level by level. In this implementation of the code, I understood how different data structures and traversal methods can be used to efficiently explore and analyze a graph.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- **Depth-First Search (DFS)** would be the most helpful algorithm. DFS starts from one vertex and explores as far as possible along one path before backtracking to the previous vertex and exploring another path. This matches the situation described in the question.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- The equivalent of **DFS in tree traversal is Pre-order, In-order, or Post-order traversal**. All of these follow the depth-first approach by exploring nodes down a branch before moving to another branch.
- DFS in Graph DFS in Tree
- A → B → D → E → C A → B → D → E → C

3. In the performed code, what data structure is used to implement the Breadth First Search?

- **Queue** is used to implement **Breadth-First Search (BFS)**. The queue follows **FIFO (First In, First Out)**, allowing BFS to visit vertices level by level.

4. How many times can a node be visited in the BFS?

- A node is **visited only once** in BFS when a visited array or similar mechanism is used. This prevents the algorithm from repeatedly visiting the same node, especially when the graph contains cycles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.